

Deep Reinforcement Learning for Traffic Light Control in Jönköping

Ben Bekir Ertugrul
School of Engineering
Jönköping University
Jönköping, Sweden

Ahmet Alper Basturk
School of Engineering
Jönköping University
Jönköping, Sweden

Abstract—TODO

I. INTRODUCTION

Rapid global urbanization and rising populations create a demand for traffic throughput optimization in order to save time, money, and resources, while increasing road safety. Traditional Traffic Signal Control (TSC) systems remain inefficient, relying on static, pre-programmed algorithms that fail to adapt to real-time traffic fluctuations. At the same time, artificial intelligence is experiencing widespread integration across decision-making domains and allows for adaptive, intelligent control systems that offer a direct path toward mitigating urban gridlock.

This paper focuses on the city center of Jönköping, Sweden, as a case study for algorithmic intersection optimization. Jönköping is chosen as its road network experiences severe bottlenecks during peak rush hours, primarily driven by roundabouts feeding directly into closely spaced intersections controlled by traffic lights. Because these existing controllers are based on non-adaptive phase-switching strategies, they are not able to mitigate the compounding queue lengths that propagate across the interconnected junctions. Implementing a Deep Reinforcement Learning (DRL) framework provides an empirical evaluation of how Deep Learning can impact multi-intersection traffic throughput.

A. Structure of this paper

TODO

II. BACKGROUND

A. SUMO

Simulation of Urban MObility (SUMO) is an open-source traffic simulation suite designed to handle large-scale road networks [1]. The platform allows for modeling of streets, vehicles, and traffic signal logics. It incorporates the Traffic Control Interface (TraCI), an API that enables live state retrieval and manipulation of traffic light phases [2]. SUMO also offers fine-grained data logging capabilities for performance metrics, making it highly relevant for intelligent transportation experiments.

B. SUMO-RL

SUMO-RL is a specialized Python library designed to instantiate reinforcement learning environments using the SUMO simulator [3]. It wraps the raw simulation variables retrieved from TraCI API into a standardized interface and provides default function for observation spaces including lane density and queue lengths, and reward functions.

C. Stable Baselines3

Stable Baselines3 offers deep reinforcement learning algorithm implementations built upon the PyTorch framework [4]. It provides optimized architectures for both value-based and policy-gradient methods, including Deep Q-Networks (DQN) and Proximal Policy Optimization (PPO), eliminating implementation bias and serving as a basis for comparative agent analysis.

III. RELATED WORKS

TODO: name 2-3 related works and quickly mention topic, method, and findings

IV. ENVIRONMENT

A. Network Generation

The urban road network was modeled following standard traffic scenario generation workflows [5]. First, the raw geographical data for Sweden was obtained from Geofabrik¹, and a custom bounding polygon was created using the GeoJSON² web interface to isolate the target study area³ shown in figure 1.

These boundaries were selected specifically because they contain the main roads in the Jönköping city center, including a total of five roundabouts. A challenging bottleneck within this area occurs along Östra Strandgatan (visible in the lower-right section of the filter), where two major roundabouts operate close together and feed directly into intersections controlled by traffic signals.

To construct the network, the command-line tools *osmium* and *osmfilter* were used to clip the Swedish map data down to the defined GeoJSON polygon. Next, irrelevant infrastructure

¹<https://download.geofabrik.de/europe/sweden.html>

²<https://geojson.io/>

³<https://github.com/benbekir/traffic-rl/blob/main/simulations/networks/jkpg/artifacts/jkpg.geojson>



Fig. 1. The GeoJSON boundary polygon (shaded in grey) defining the geographic scope of the Jönköping road network.

was filtered out using OpenStreetMap highway attributes, including:

- footway
- path
- pedestrian
- steps
- cycleway
- bridleway

Finally, the compiled map data was processed using the SUMO *netconvert* utility to generate the executable simulation network. The SUMO network is visualized in figure 2.



Fig. 2. The finalized road network displayed within the SUMO Netedit interface, highlighting the target traffic light placements.

B. Simulation Scope and Limitations

The generated environment captures real-world lane geometries, intersection configurations, speed limits, and traffic light locations. However, certain simplifications were enforced to bound the complexity of the experiment. First, the simulation models passenger vehicle traffic exclusively and disregards pedestrians. Consequently, pedestrian-actuated crossing signals operate on semi-realistic default patterns rather than

dynamic activation. Second, urban bus transport, which contributes heavily to public transit volume in central Jönköping, was excluded from the routing profile. We acknowledge that these abstractions deviate from real-world conditions, and the simulation should be interpreted as a comparative baseline for algorithmic performance rather than an exact replica of local traffic behavior.

C. Traffic Demand Modeling

Due to the absence of publicly available traffic assignment zones or quantitative origin-destination flow data for Jönköping, a deterministic, observation-driven demand model was engineered. Relying on entirely randomized vehicle insertion across the network is unrepresentative of real traffic behavior, as it distributes vehicle density uniformly across minor residential streets and major roads alike.

Therefore, to ensure a realistic flow, an origin-destination matrix was manually constructed based on our own observations. This matrix maps commuter behaviors by defining vehicle volumes, timestamps, and network entry and exit edges. To maintain realistic routing, traffic flows are restricted exclusively to boundary edges of the network. The SUMO *duarouter* utility is then used to build the demand file and randomize vehicle routes by computing the shortest-path between the specified points and adding minor variations.

To simulate high-pressure gridlock conditions, the 8,400-second simulation timeline incorporates two distinct peak rush-hours:

- **Baseline Phase (Seconds 0–2000):** Operates under standard, off-peak baseline volumes.
- **Morning Peak (Seconds 2000–5000):** Simulates morning commuting patterns, where the majority of vehicles travel from east to west.
- **Evening Peak (Seconds 5000–8400):** Simulates evening commuting patterns, where the majority of vehicles travel from west to east.

D. Markov Decision Process Formulation

To facilitate reinforcement learning, the traffic signal control problem is formalized as a Markov Decision Process (MDP) consisting of a state observation space, a discrete action space, and a scalar reward function mapped to each agent.

1) *Observation Space:* The observation vector for each traffic signal agent provides intersection's operational state. The state is represented as a multi-dimensional vector structured as follows [6]:

$$O = [P_{\text{one-hot}}, M_{\text{green}}, D_1, \dots, D_n, Q_1, \dots, Q_n] \quad (1)$$

Where:

- $P_{\text{one-hot}}$: A one-hot encoded vector representing the currently active green phase configuration.
- M_{green} : An indicator specifying whether the minimum green phase duration has elapsed.
- D_i : The normalized vehicle density of incoming lane i , calculated by dividing the current number of present

vehicles by the maximum physical vehicle capacity of that lane.

- Q_i : The normalized queue length of incoming lane i , defined as the number of halted vehicles (traveling at a velocity below 0.1 m/s) divided by the total lane capacity.

2) *Action Space*: The action space is modeled as a discrete set where each action corresponds to a specific green phase configuration at the intersection [7]. Every 5 seconds, the agent evaluates the state observation and selects the next phase configuration. To maintain traffic safety and mimic physical infrastructure constraints, any action that triggers a change in the active phase configuration is automatically prepended by a yellow phase lasting a fixed 2 seconds.

3) *Reward Function*: While the underlying SUMO-RL framework supports various native reward formulations, including changes in cumulative vehicle delay, average velocity metrics, pressure optimization, and carbon dioxide emissions, this study implements the specialized "queue" reward function [8]. The reward is mathematically defined as the negative sum of all queued vehicles across all incoming lanes feeding into the intersection agent [9]:

$$R_t = - \sum_{i=1}^n \text{HaltingVehicles}_i(t) \quad (2)$$

V. EXPERIMENT SETUP

A. Reinforcement Learning Agents

In this study, we evaluate and compare two distinct classes of DRL architectures: a value-based DQN method and a policy-gradient PPO method.

1) *Deep Q-Networks (DQN)*: Deep Q-Networks combine traditional Q-learning with deep neural networks to approximate the optimal action-value function. DQN is an off-policy algorithm that stores past experiences in a large replay buffer, allowing it to sample random, uncorrelated historical batches to stabilize training. During evaluation, the agent greedily selects actions with the estimated Q-values, picking the highest scoring traffic phase configuration for a given state.

2) *Proximal Policy Optimization (PPO)*: Proximal Policy Optimization is an on-policy, actor-critic framework that directly optimizes the agent's control policy. Instead of assigning strict value scores to actions, PPO maps states directly to a probability distribution over the available traffic phases. To decrease the chance of catastrophic forgetting off destabilizing gradient steps during chaotic traffic conditions, PPO clips its policy updates within a safe bound. It learns strictly from current environment steps, discarding data after each update phase.

B. Hyperparameter Configurations and Training Protocol

Both algorithms were initialized using parameter baselines optimized over a preliminary 100-episode training phase. Using these optimized baselines, a curriculum learning strategy was deployed to progressively scale the traffic demand density across three distinct multiplier steps: 0.75, 0.875, and 1.0. For the final evaluation, the DQN agent was trained for 100

episodes per curriculum phase to maximize historical experience accumulation. However, initial trials showed that PPO's performance degraded with the extended training horizon. To mitigate this policy degradation, the training duration for the PPO agent was shortened to 50 episodes per curriculum phase. The finalized hyperparameter configurations utilized for this execution are compiled in Table I.

TABLE I
HYPERPARAMETER CONFIGURATIONS FOR DQN AND PPO AGENTS.

Algorithm	Hyperparameter	Value
DQN	Learning Rate	1×10^{-3}
	Buffer Size	500,000
	Batch Size	128
	Exploration Fraction	0.1
	Initial Epsilon (ϵ_{init})	1.0
	Final Epsilon (ϵ_{final})	0.05
PPO	Learning Rate	1×10^{-4}
	Rollout Horizon (n_{steps})	512
	Batch Size	256
	Epochs per Update (n_{epochs})	10
	Entropy Coefficient (ent_coef)	0.2
	Clip Range (ϵ_{clip})	0.3

The DQN agent implements an ϵ -greedy exploration strategy where the exploration rate linearly decays from an initial value of 1.0 down to a final value of 0.05 across the first 10% of the total training steps. It uses an experience replay memory buffer of 500,000 steps to maintain a large historical distribution of past traffic patterns, sampling mini-batches of 128 experiences per optimization step.

Conversely, the PPO agent operates under a more conservative learning rate of 1×10^{-4} to stabilize gradient updates. The agent collects experiences over 512 steps before performing 10 optimization epochs across mini-batches of 256 steps. To preserve policy exploration, an elevated entropy coefficient of 0.2 was selected. Furthermore, a policy clipping threshold of 0.3 enforces strict bounding constraints on the policy updates, preventing catastrophic divergence when the agent is exposed to severe rush-hour gridlock configurations.

Every configuration was evaluated across 10 independent random seeds to guarantee statistical significance and capture structural performance variance.

VI. RESULTS

TABLE II
PERFORMANCE COMPARISON ACROSS 10 RUNS (AVERAGED).

Metric	Baseline	DQN	PPO
Completed trips	5,837	5,192	5,443
Avg. waiting time	97.4s	12.5s	35.1s
Avg. time loss	199.4s	45.2s	89.3s
Avg. stops per vehicle	6.48	1.11	2.97
Avg. speed	6.73 m/s	8.81 m/s	8.10 m/s
Total waiting time	568,322s	64,689s	201,287s
Simulation end time	8,398s	8,398s	8,157s

TABLE III
IMPROVEMENT OVER BASELINE (10-RUN AVERAGE).

Metric	DQN	PPO
Waiting time	-87.2%	-64.0%
Time loss	-77.3%	-55.2%
Stops per vehicle	-82.8%	-54.2%
Average speed	+30.9%	+20.5%
Completed trips	-11.0%	-6.8%

VII. CONCLUSION

REFERENCES

- [1] P. Alvarez Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner, “Microscopic traffic simulation using sumo,” in *2019 IEEE Intelligent Transportation Systems Conference (ITSC)*. IEEE, November 2018, pp. 2575–2582. [Online]. Available: <https://elib.dlr.de/127994/>
- [2] “TraCI Documentation,” <https://sumo.dlr.de/docs/TraCI/>, 2026, accessed: 2026-05-30.
- [3] L. N. Alegre, “SUMO-RL,” <https://github.com/LucasAlegre/sumo-rl>, 2019.
- [4] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: <https://jmlr.org/papers/v22/20-1364.html>
- [5] D. A. Guastella, E. Montero-Porras, A. Morales-Hernández, and G. Bontempi, “Traffic modeling with sumo: a tutorial,” 2025. [Online]. Available: <https://arxiv.org/abs/2304.05982>
- [6] “SUMO-RL Default Observation,” <https://lucasalegre.github.io/sumo-rl/mdp/observation/>, 2026, accessed: 2026-05-30.
- [7] “SUMO-RL Actions,” <https://lucasalegre.github.io/sumo-rl/mdp/action/>, 2026, accessed: 2026-05-30.
- [8] “Enumeration of default reward functions,” https://github.com/LucasAlegre/sumo-rl/blob/main/sumo_rl/environment/traffic_signal.py#L347, 2026, accessed: 2026-05-30.
- [9] “Implementation of the queue reward function,” https://github.com/LucasAlegre/sumo-rl/blob/main/sumo_rl/environment/traffic_signal.py#L321, 2026, accessed: 2026-05-30.